

Lab 3: Architectural Pattern Selection – Justification

Public Bus Live Tracking and Crowding Estimator System

Architecture Selection

We chose Microservices Architecture for the Public Bus Live Tracking and Crowding Estimator System.

Layered and Client-Server styles were also evaluated. Layered architecture is simple but scales as one block, and Client-Server relies on a single central server that becomes a bottleneck and single point of failure. Microservices best fits a system whose parts receive very different loads and must keep running when one part fails.

Two Reasons for the Choice

1. **Different workloads per feature.** The Bus Tracking Component receives a GPS update from every bus every few seconds, while the Crowding Estimator processes passenger-count events at stops, and the Fleet Monitoring & Alert Component runs only on threshold rules. Each is a separate component behind its own interface, so it can be developed, deployed and scaled on its own.
2. **Fault isolation and replaceability.** If the ticketing sensor or the Crowding Estimator fails, passengers still get live locations and ETAs, because Bus Tracking and ETA Management do not depend on it. Any component (for example, a better ETA algorithm) can be swapped out as long as it keeps the same provided interface, as in the component diagram.

Security Advantage

Each service exposes only narrow, well-defined interfaces, which limits the attack surface. The Passenger App can reach only the read-only ETA Information and Crowding Status interfaces. The Database Component is reachable only through the Data Access API and never directly from clients. Alert Information is sent only to the authenticated Fleet Controller Dashboard, and all service-to-service and client traffic uses HTTPS/TLS with token-based authentication. A breach of one service therefore does not expose the others. Crowding is estimated as an anonymous count, so no personal passenger data is stored.

Performance Benefit

Independent scaling gives the main performance gain. During peak hours more instances of the Bus Tracking and ETA Management components can be started to handle heavy GPS traffic, without duplicating the whole system. Lightweight messaging (MQTT for GPS and sensor feeds) and cached ETA results keep the end-to-end delay from a bus position update to the passenger's screen low, so live tracking stays responsive.

Trade-off Acknowledged

Microservices add network latency and operational complexity. This is limited here by keeping only five focused services with a shared data store behind a single Data Access API; per-service databases could be introduced later if data volume grows.